

Using Reinforcement Learning and Algorithmic Techniques to Develop Strategies for Tournament-Style Backgammon Matches

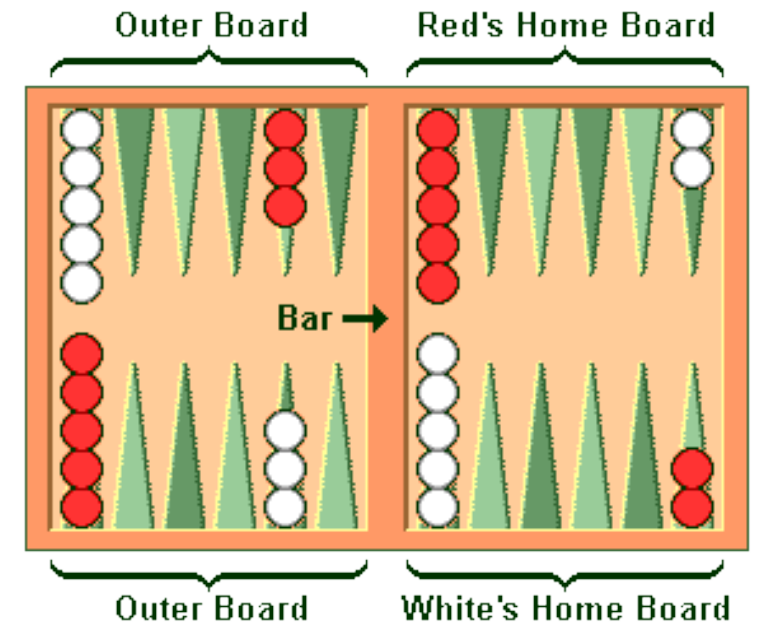
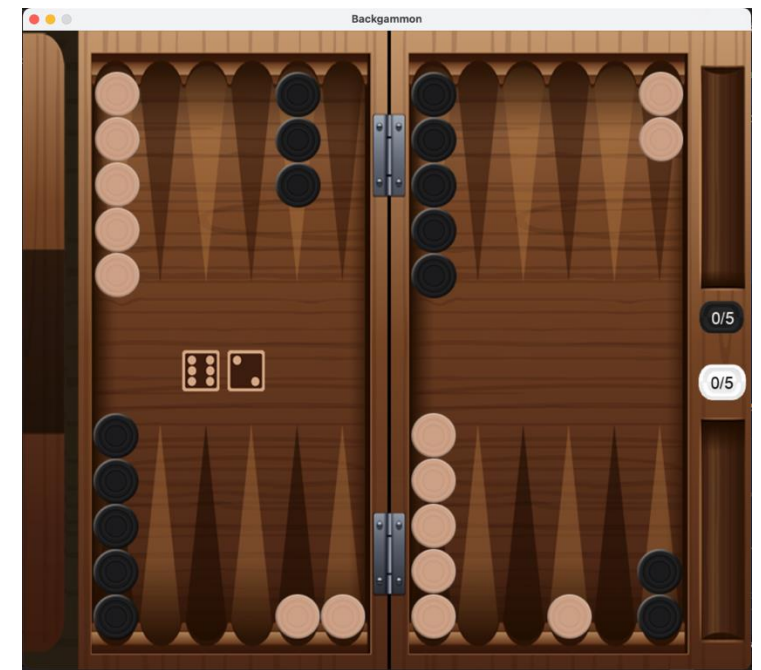
Will Farmer

Computer Science (Level 3)

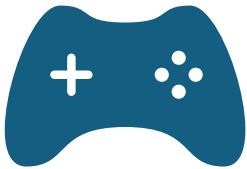
09/05/2024

What is Backgammon?

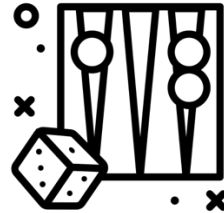
- A Two-player zero-sum probabilistic game
- Both players attempt to move their pieces around the board into their 'home'
- First player to move all their pieces into their home and bear the pieces off the board wins
- Moves are dictated by the dice rolls
- Pieces can be "hit" off the board, where they must re-enter before any other piece can be move
- A blot is a single piece on a point and can be hit off
- An anchor is when >1 pieces occupy a point
- A prime is 4 or more consecutive anchors
- Pip count is the total distance all a player's pieces must travel to win the game
- Bear-in: moving pieces into home. Bear-off: moving pieces off the board



Motivation



All popular backgammon algorithms focus on winning individual games



Backgammon is played in a first-to format, where the victor is the first player to reach the score limit



You can strategize for a better point return by using the Doubling Cube, or aiming to win by gammon or backgammon, which yield additional points

Motivation

Tesauro's TD-Gammon is almost 20 years old and is still the leading Backgammon AI

A reinforcement learning model focused on winning individual games

Uses heuristic doubling algorithm

Model didn't include information about backgammons

Do agents designed to maximise PPG differential instead of win rate perform better in tournament-style matches?

Deliverables



Basic

Random Agent
Robust Game Environment
Graphical User Interface (GUI)



Intermediate

Greedy Agent
Genetic Agent
Expectimax Agent



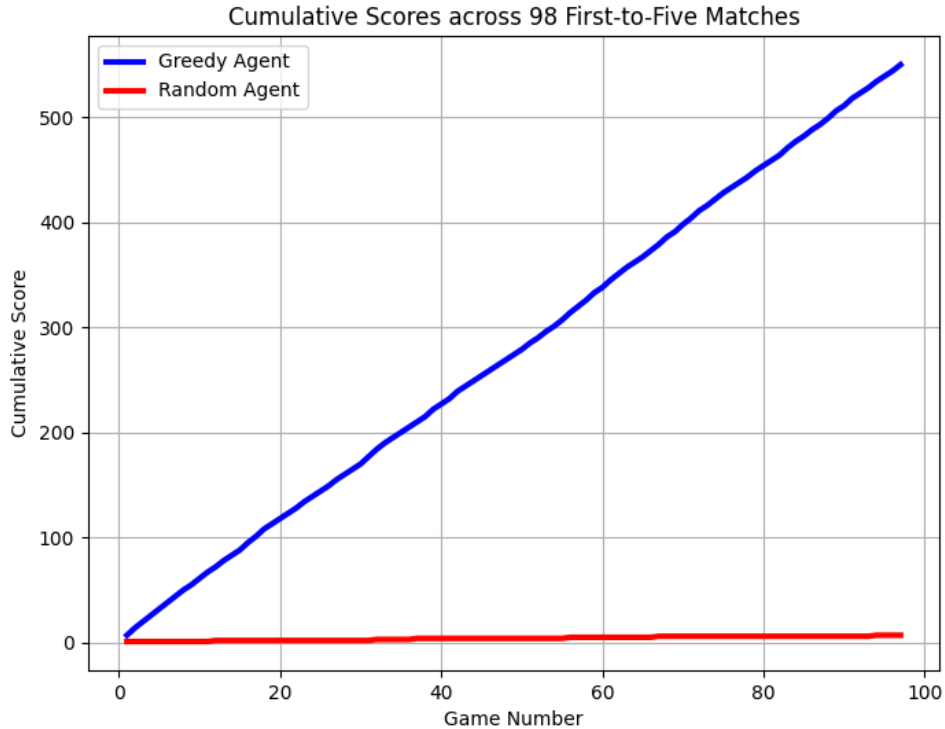
Advanced

Reinforcement Learning Agent
Machine Learning Agent
Agent that changes strategy based
on state of the game

Robust Game Engine



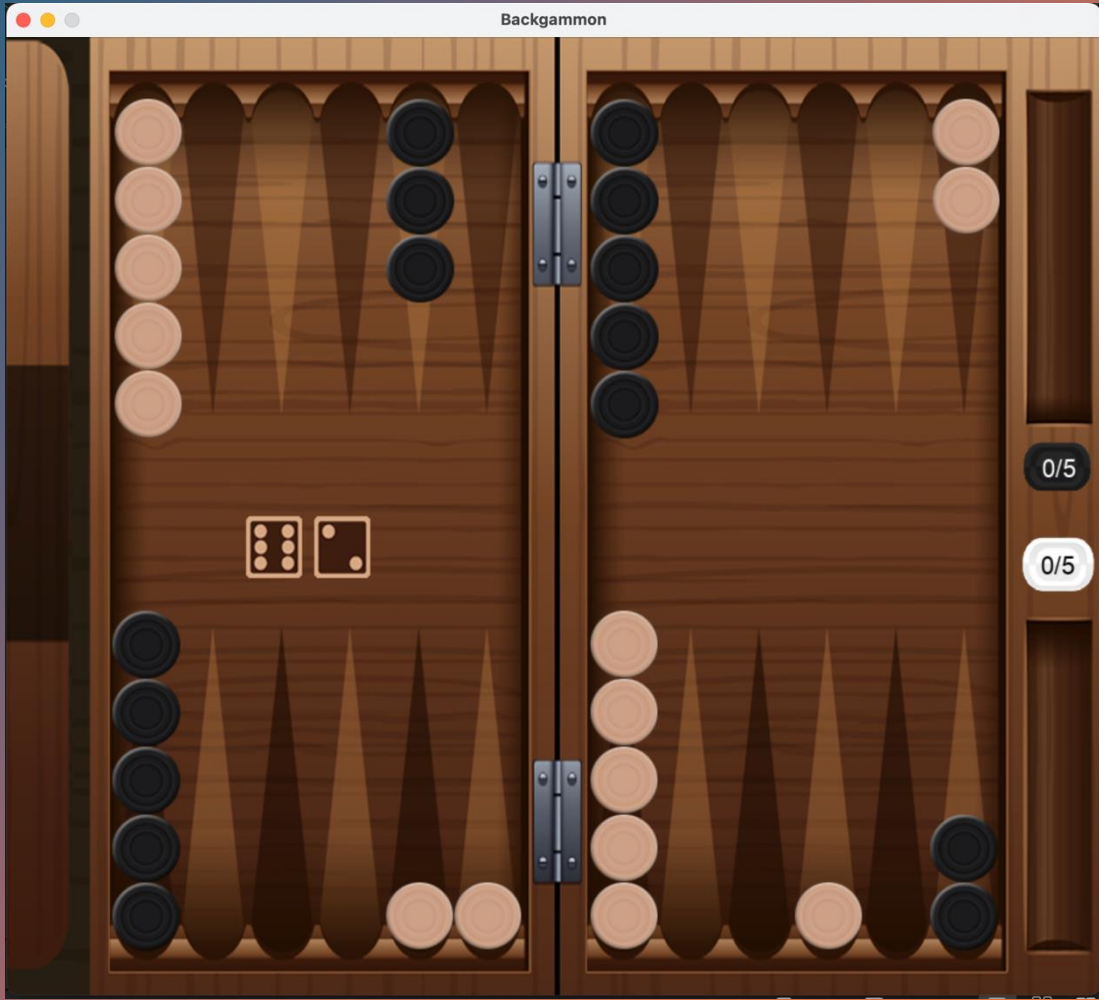
Random Agent



Final Score: 545-0

Rounded Average score: 6-0

- Generates random integers until they form valid moves
- Developed to test the robustness of the game engine and provide a beginner-level opponent for user and baseline testing
 - Helped locate errors with rolling doubles, entering and bearing off
- Loses convincingly against any opponent



GUI

- Extremely useful for viewing games and analysing decisions
- This has been vital in debugging various issues with agents
 - Especially with the Greedy Agent
- Allows humans to play agents without knowledge of backend program, paramount for user testing
- Allows for direct comparison between different agents

Greedy Agent

Scores each possible resulting board

- Utilises 17 hard-coded weights and nested conditional statements

In the case where multiple resulting boards have the same score, a tiebreaker occurs.

- Tiebreaker was introduced to remedy unintended bias

Always chooses the predefined best move

- No lookahead beyond end of agent's turn

Completed before all other intermediate deliverables

- They would require features present in the Greedy Agent's code

Genetic Agent

- All the nested conditions from the greedy agent were flattened
- Initial population has randomly generated weights
- Fitness is determined by first-to-five matches of the individual vs the greedy agent:

$$fitness = \begin{cases} Score_i \div Score_g & \text{if } Score_g > 0 \\ Score_i + 1 & \text{otherwise} \end{cases}$$

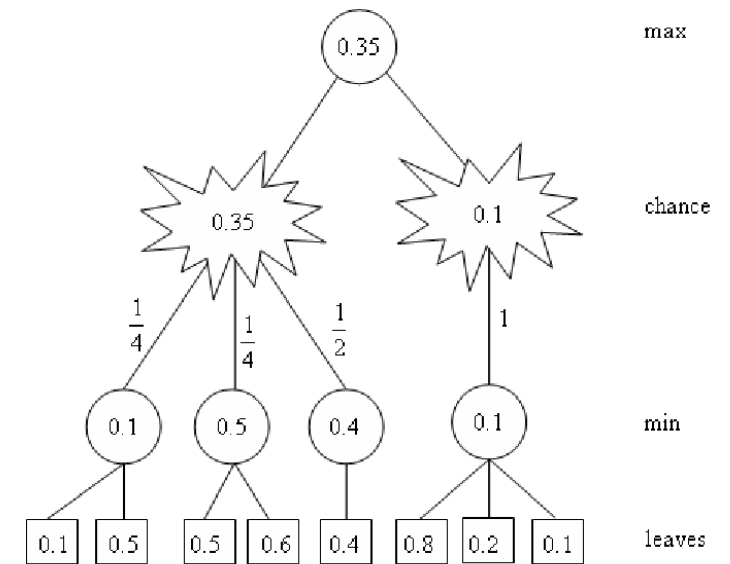
- All individuals with a fitness greater than 5 then coevolve against each other in first-to-25 matches until only one remains.
- Remaining individual is then chosen to be the genetic agent



Example Coevolution tournament bracket

Expectimax Agent

- The agent checks which move is theoretically worst for the opponent and plays it
- Uses the greedy agent's evaluation function to score each of the opponent's possible follow-up moves
- Then chooses the move that minimizes the score of the opponent's follow-up



Tree from perspective of the AI

Comparison of Intermediate Agent Performance

Expectimax vs
Greedy across 5000
games

- 3157-3898 final score
- 2436-2574 wins.

Expectimax vs
Genetic across 1000
games

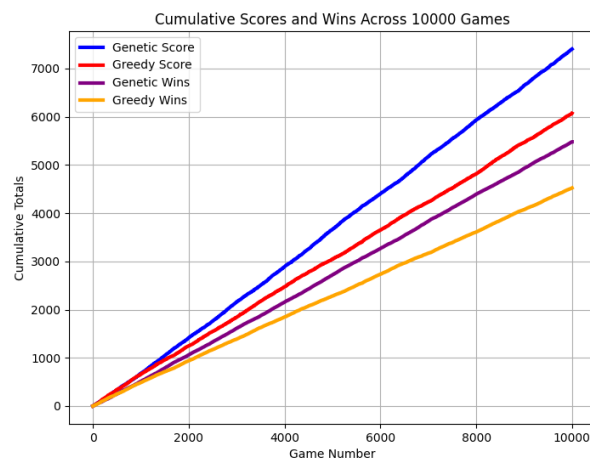
- 567-898 final score
- 431-569 wins

Genetic vs Greedy
across 10,000
games

- 7401-6068 final score
- 5478-4522 wins

Visualisation of Head-to-Head Results

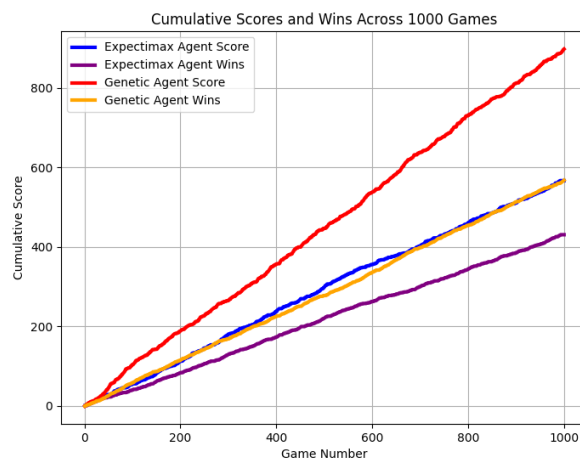
Genetic vs Greedy



Genetic agent +0.133ppg

Genetic wins 54.8%

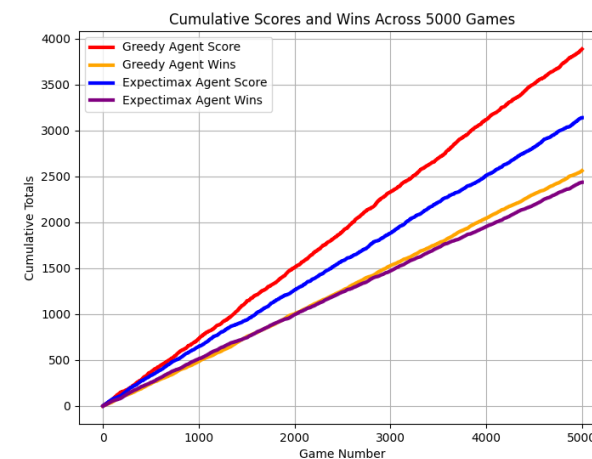
Genetic vs Expectimax



Genetic agent +0.331ppg

Genetic wins 56.9%

Greedy vs Expectimax



Greedy agent +0.148ppg

Greedy wins 51.5%

Adaptive Agent

Utilises different strategies at different phases of the game

Turn 1 uses the Genetic agent's move-selection process as it has best opening & counter-opening move selection

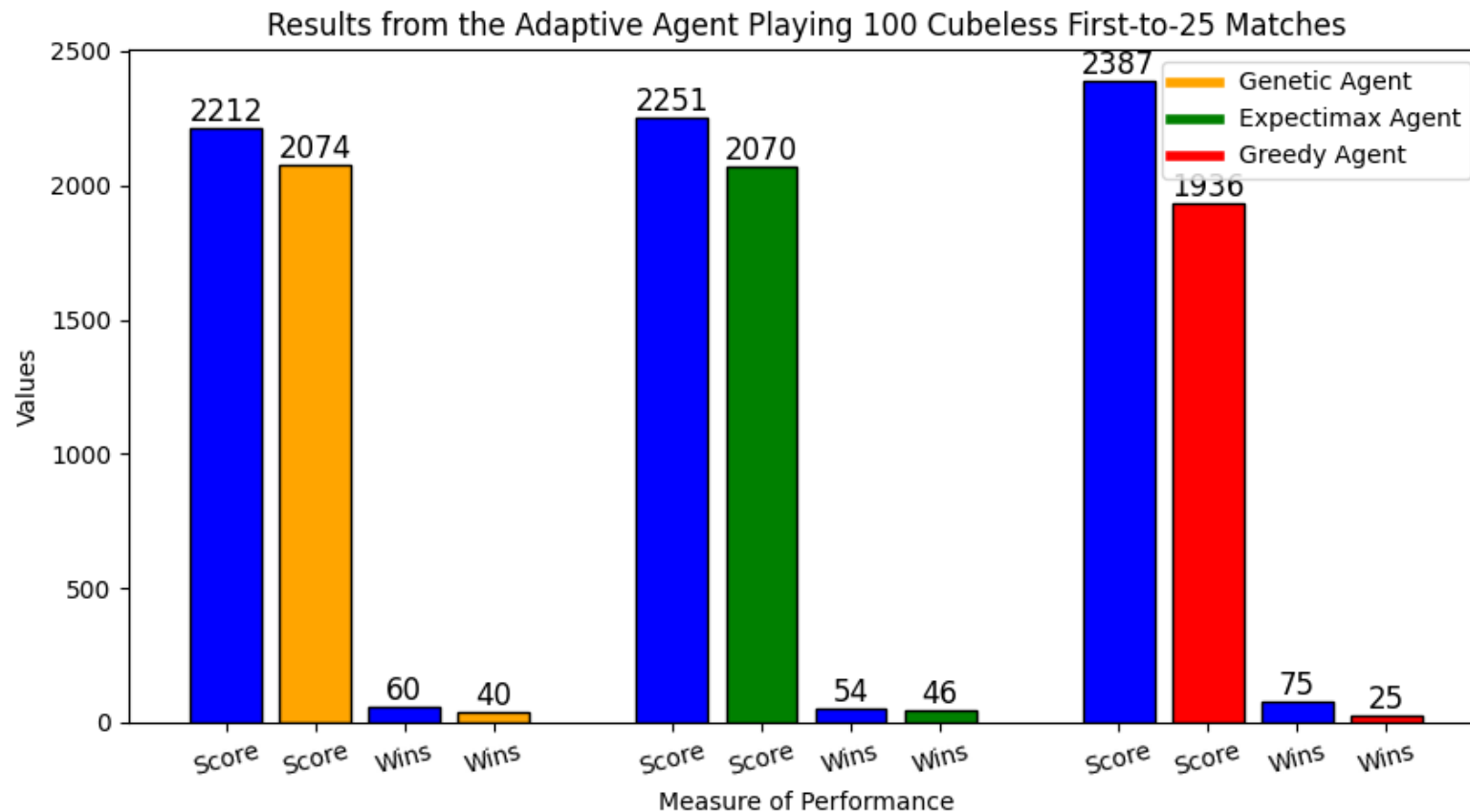
In a bear-off race uses Matussek's game-winning-chance formula for optimally bearing-off and positioning checkers

If high risk of being gammoned, moves furthest piece back

Otherwise uses 30-parameter equity calculation to determine board strength and game-winning-chance

If equity tie, uses one-move lookahead to probabilistically make best move

Performance of the Adaptive Agent



Adaptive Agent Doubling



Uses a combination of theoretical formulae and genetic methods to make doubling decisions



Played hundreds of games against the intermediate agents and identified which doubling points maximized PPG differential



To improve generalizability another co-evolution tournament took place to identify which of the high-performing doubling points were the best



When in a race position, the agent uses Keeler and Spencer's optimal doubling points in conjunction with Matussek's regression formula

Deep Agent



Utilises Deep Neural networks



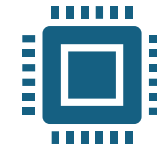
Trained using Supervised Learning on expert equities

GNUBG



Was initially trained to output an estimated value for the state on Mean Squared Error loss

Small inaccuracies led the model to perform poorly in practice even with low MSE loss



Later trained to output a preference between two board states and was trained on Binary Cross Entropy loss

Huge increase in performance

Deep Agent Iterations

1

Version 1 Initially had 1 general-purpose model but had identifiable weaknesses

- Couldn't tell difference between race and contact positions
- Didn't understand value of bearing pieces off

2

Version 2 was trained for an additional 100 epochs

- Decrease in performance as model overfitting to training set

3

Versions 3 and 3.5 contained 3 specialist networks

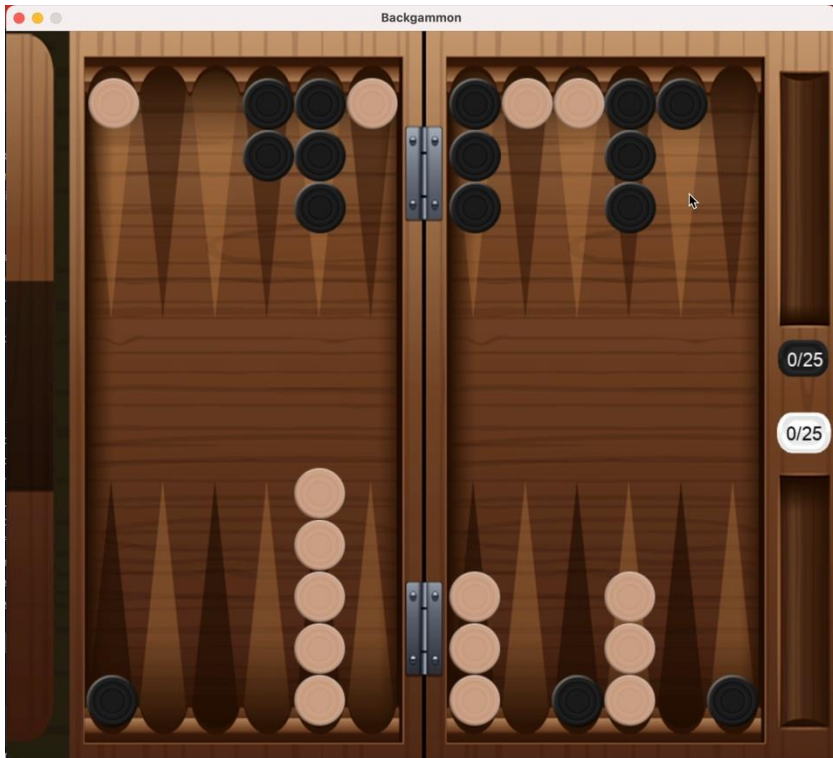
- Performance decreased as contact specialist could not plan ahead

4

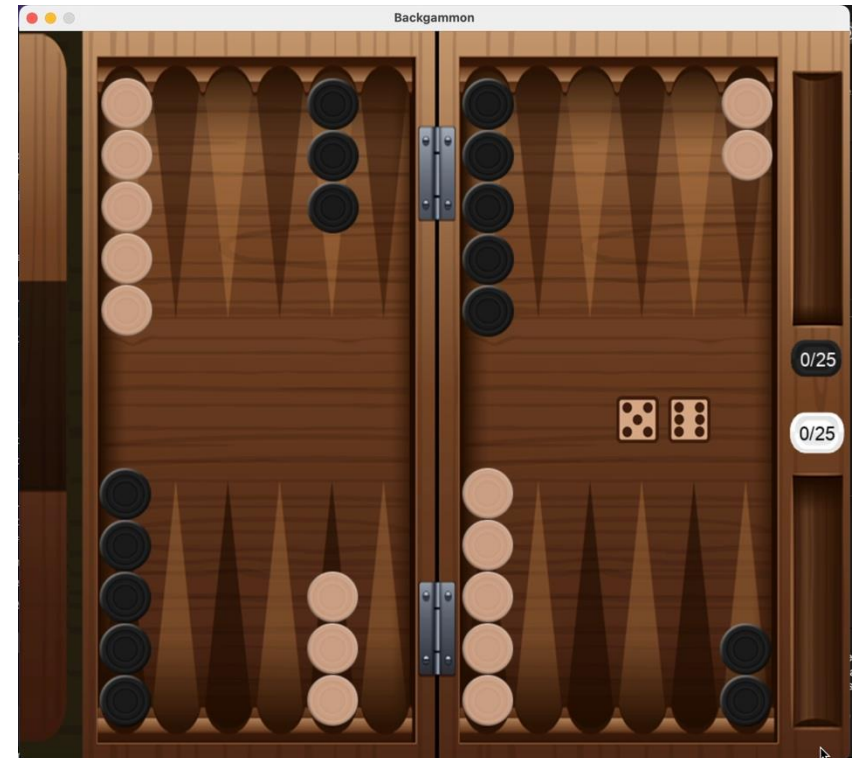
Versions 4 and 4.5:

- 3 networks for different types of position
- Initial general-purpose network for contact positions
- Bear-in network for getting pieces home when in a race
- Bear-off network for bearing pieces off the board when home

Gameplay

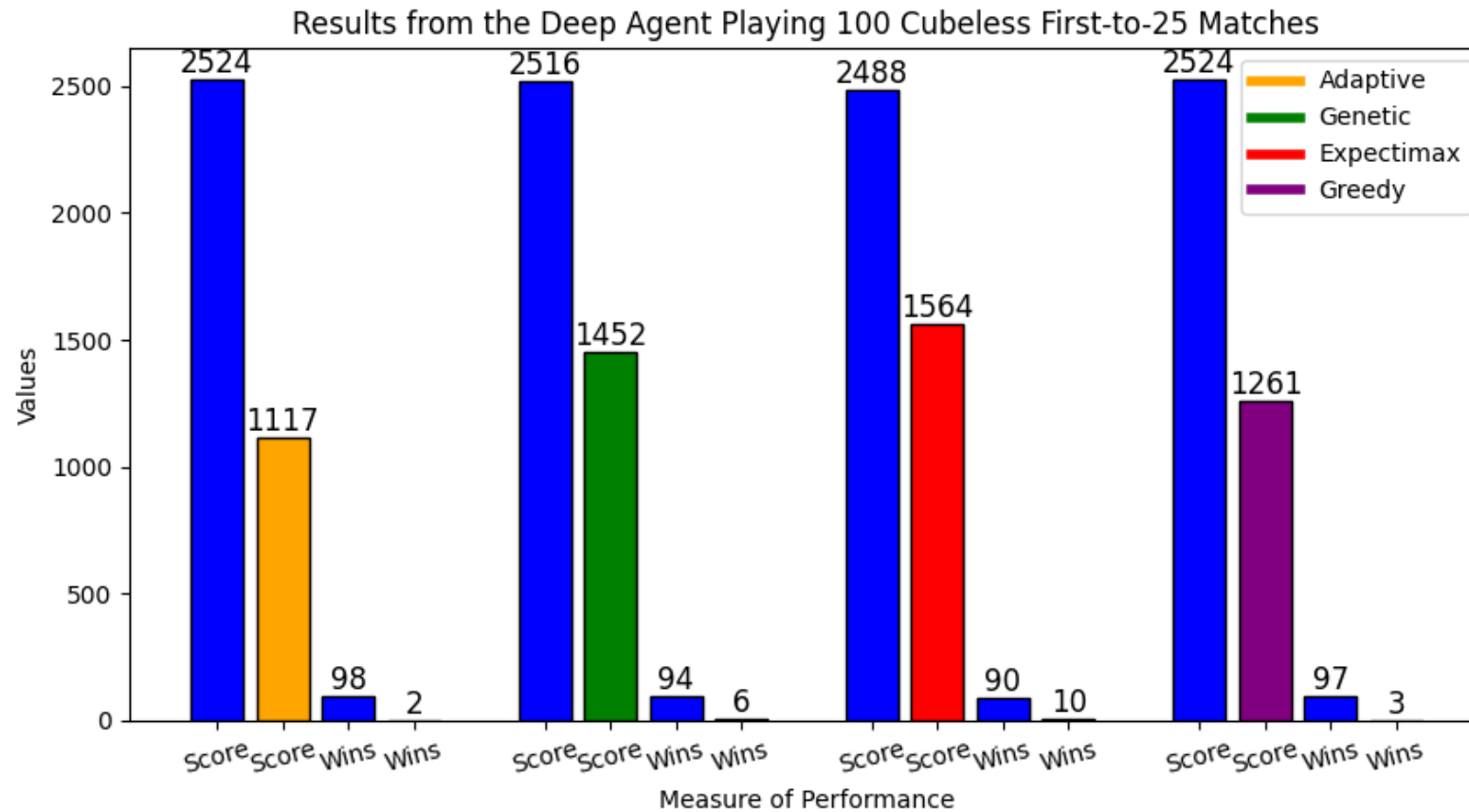


Great contact play



Opening move with roll 5-6

Deep Agent Head-to-Head Performance



Potential Improvements to Deep Agent

Makes mistakes when it sees its guaranteed to win or lose

- Was trained on expert-generated equities
- When it thinks its guaranteed to lose, it effectively gives up
 - Sometimes loses by backgammon when could have easily just been a gammon
- Retrain model with handcrafted examples for guaranteed-loss positions

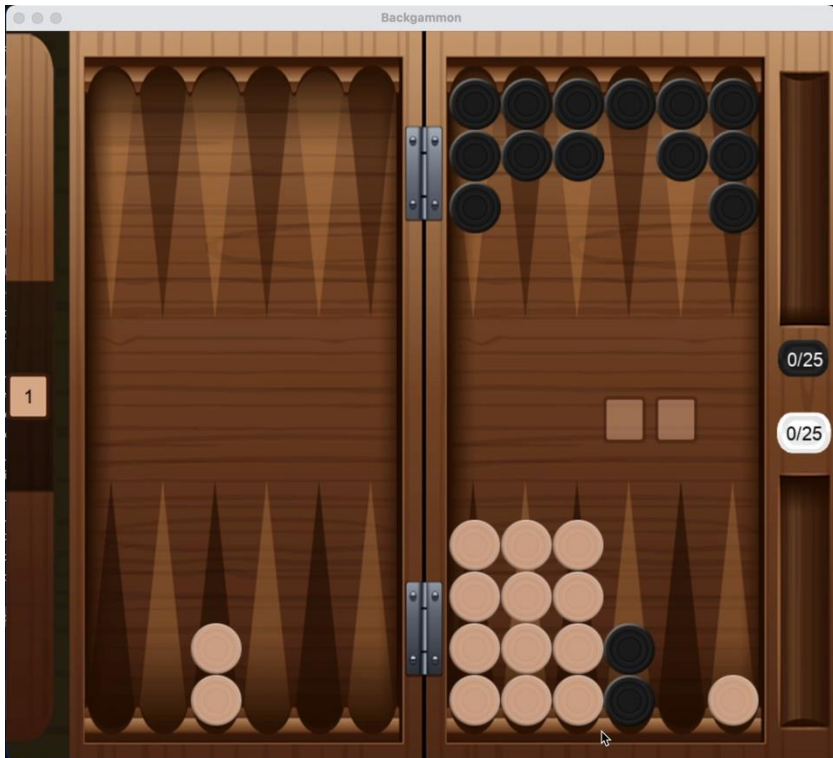
Blunders on two of the best opening rolls

- Could make a specialist opening move network
- Could even just hardcode the best move to play as there are only 15 possible opening rolls
- Most opening rolls are accurate though

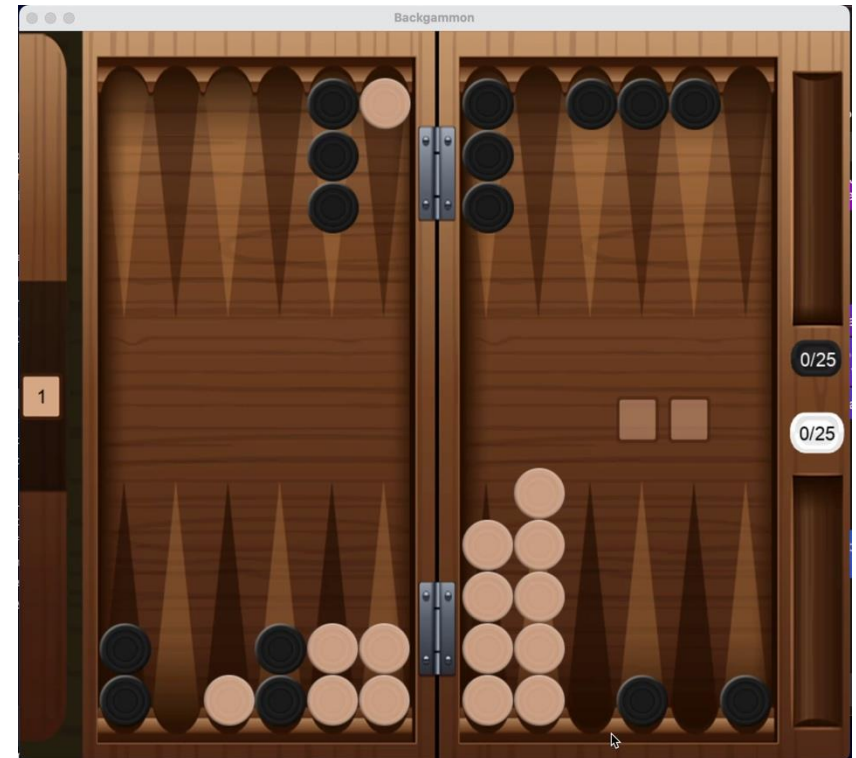
Deep Agent Doubling

Models	Structure	Description	Justification
Contact Accept + Offer	291-12-1	Same board state and precomputed features as move-selection networks. Added units represented number of borne off pieces for each player.	• Binary classification task • Explicit representation of borne-off pieces as network would sometimes choose to double even if opponent has borne off many more pieces
Race Accept + Offer	265-12-1	Same as Contact networks except removed all precomputed features that are only relevant to contact position	• Success with contact models • Many of the precomputed features are irrelevant in a race so were removed to prevent distraction and computation time

Doubling Decisions

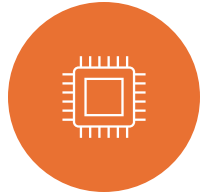


Rejects Double



Accepts double

Reinforcement Agent



Uses Temporal-Difference(λ) learning



Trained in hundreds of thousands of games playing against itself



Rudimentary board encoding without any precomputed features



Uses 2-ply selective lookahead to minimise the impact of counterplays while maximising equity

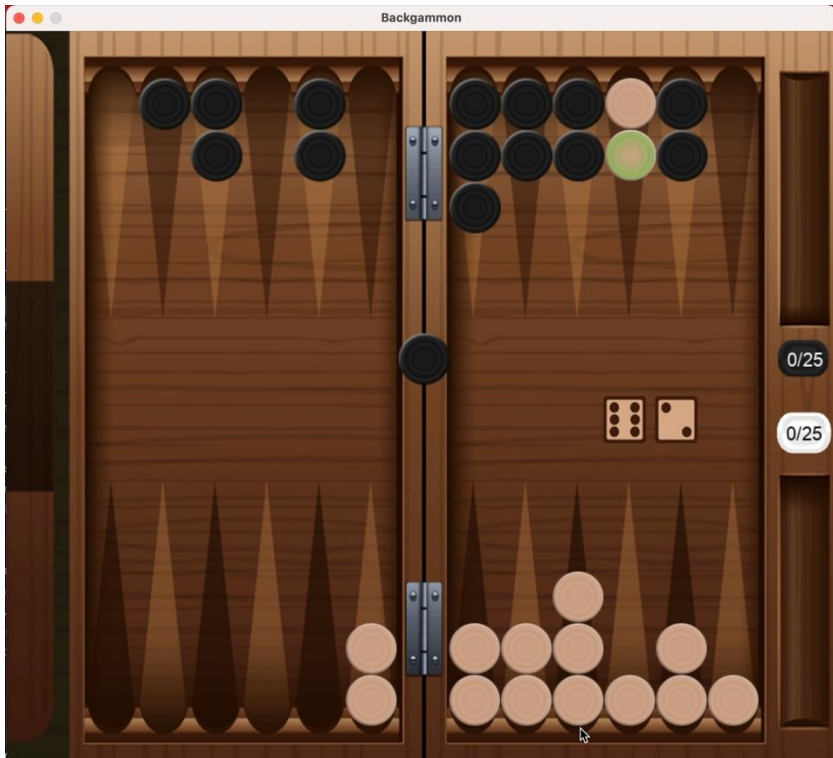


Uses Deep Agent's doubling strategy

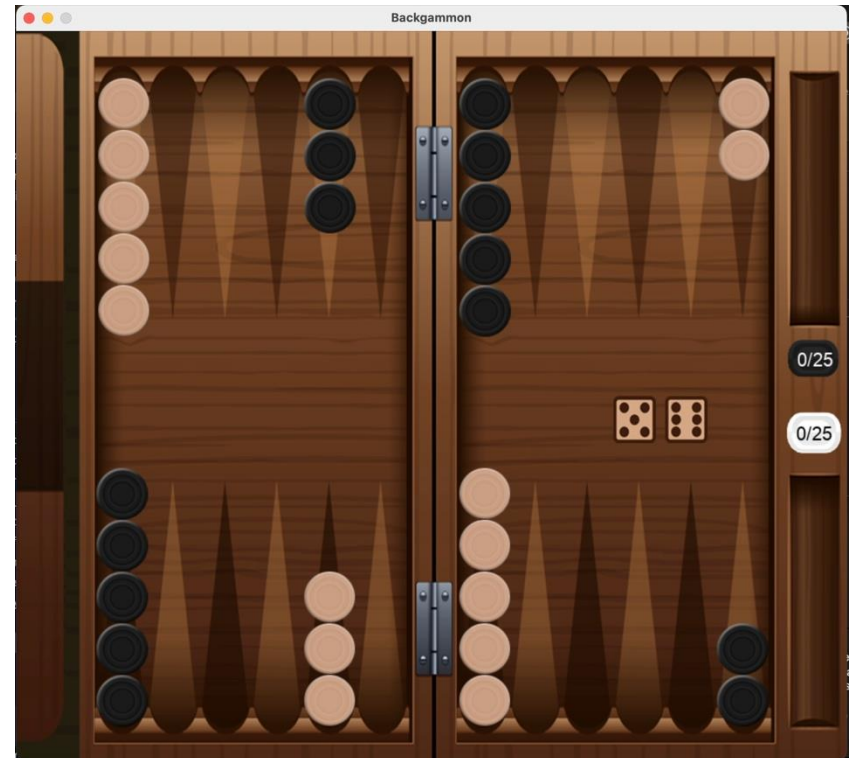


Calculates an expected value by estimating the probability of each terminal state occurring and chooses the move that maximises this value

Gameplay Highlights

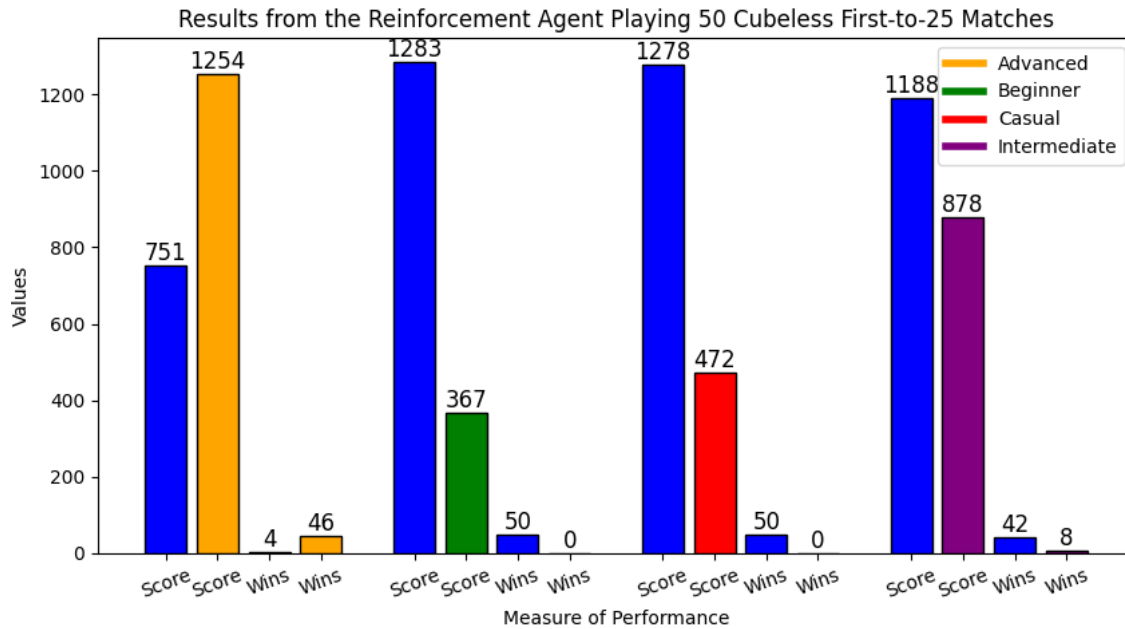


Amazing positional awareness

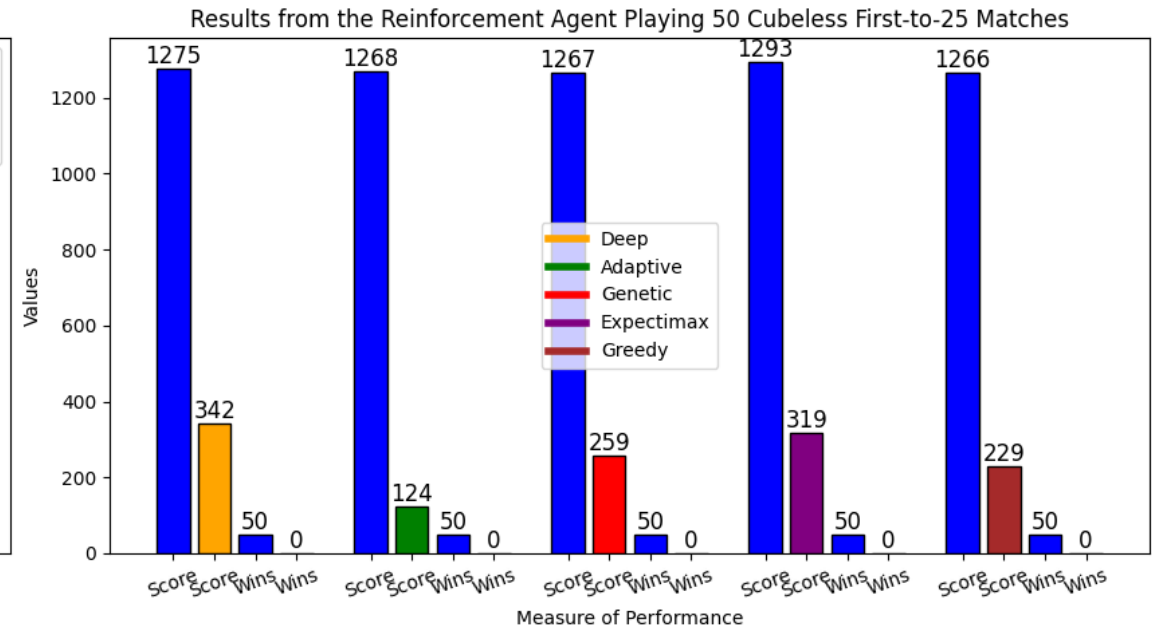


Best opening move on 6-5 roll

Performance of the Reinforcement Agent



Achieves advanced-level play



Far superior to all other agents presented in this project

Potential Improvements to Reinforcement Agent

Extend

Extend lookahead to 3-ply

- This would likely improve performance in bear-off races where the opponent's actions no longer affect the AI

Include

Include precomputed features in board encoding

- This caused an enormous boost in performance between TD-Gammon 1.0 and TD-Gammon 2.0

Make

Make specialist network for racing positions

- Could generate random racing board states and set the agent to play against itself with those board state chosen as the starting board
- Specialist racing networks boosted performance in Deep agent

Evaluation of Agent Doubling Strategies

- Agents on average perform better in race positions than contact positions and when offering the cube in comparison to accepting the cube
- Offering a double is simpler: it's easy to see if you're in a winning position
- Accepting/rejecting the cube is more complex: not just about if you're losing, but by how much you're losing by and what the likelihood you'll be able to turn the game around is

	Deep/Reinforcement	Adaptive	Intermediate
Contact Accept	74.59%	63.51%	39.29%
Contact Offer	93.39%	85.06%	75.63%
Race Accept	89.32%	86.69%	84.06%
Race Offer	92.31%	86.80%	74.81%

So... do agents designed to maximise PPG differential instead of win rate perform better in tournament-style matches?

- The results indicate that they do
- TD-Gammon 1.0 used a very similar board encoding to the Reinforcement agent and only achieved intermediate-level play
 - Reinforcement agent attains weak-advanced or advanced-level play
- The Deep agent has better move accuracy than Neurogammon on unseen board states
 - Contact positions are majority of contact positions though, so may perform worse than statistics suggest in head-to-head games
- Doubling strategies help further boost PPG differential

Position	Deep Move Accuracy	Neurogammon Move Accuracy
Contact	66%	69%
Bear-in	80%	63%
Bear-off	95%	83%